PROJECT REPORT

WiFi Home Control System using ESP32-S3

A relay controller with a phone web interface and voice commands

Name: _____

Roll number: _____

Department: _____

Guide: _____

Date: _____

Contents

1. Abstract 3
2. Introduction and objectives 3
Objectives 3
3. Components used 3
Software tools 3
4. System design 4
Network modes 4
5. Circuit connections 4
Connecting a load to a relay 4
6. Software design 5
6.1 Firmware structure 5
6.2 Web interface 5
6.3 Request interface 5
6.4 Voice and typed commands 6
7. Working of the system 6
8. Testing and results 6
9. Problems faced and solutions 7
10. Limitations 7
11. Safety 7
12. Future scope 7
13. Conclusion 8

1. Abstract

This project builds a low-cost WiFi controller that switches four electrical devices from a phone. An ESP32-S3 development board drives a four-channel relay module. The board runs a small web server, and a phone opens a control page in its browser, with no app to install. The page shows one tile per device. A tile can be tapped to switch the device, and spoken or typed commands such as "fan on" or "bell" are also understood.

The first switch rings a bell for a fixed 1.5 seconds and then switches itself off, the second switches a fan, and two further switches are kept for later use. The board makes its own WiFi network so it works without a router, and it can also join a home or college WiFi network.

2. Introduction and objectives

Switching appliances by hand is inconvenient when the switch is far away or when something must be triggered briefly, such as a bell. Commercial smart switches solve this but cost more and often depend on a vendor's cloud service. A microcontroller with WiFi gives the same function at a small cost and keeps everything on the local network.

Objectives

- Switch four relays from a phone using only a web browser.
- Make the bell ring for a fixed short time and never stay on.
- Accept voice and typed commands in addition to taps.
- Work with no router (own WiFi network), and also on an existing WiFi network.
- Keep all software on the board itself, with no cloud service.

3. Components used

Component	Purpose
ESP32-S3 DevKitC-1	Microcontroller with built-in WiFi. Runs the firmware and the web server.
4-channel relay module (5 V, active-LOW)	Electrically isolated switches that handle the load power. Controlled by the ESP32's pins.
USB cable (data type) and 5 V supply	Programming the board and powering it (a phone charger of 1 A or more).
Bell, fan (loads)	The devices that are switched.
Phone or laptop	Runs the web page in a browser.

Software tools

- PlatformIO in Visual Studio Code, with the Arduino framework for ESP32.
- Libraries: WiFi, WebServer and ESPmDNS from the ESP32 Arduino core.
- The web page uses plain HTML, CSS and JavaScript stored inside the firmware, with no external files.

4. System design

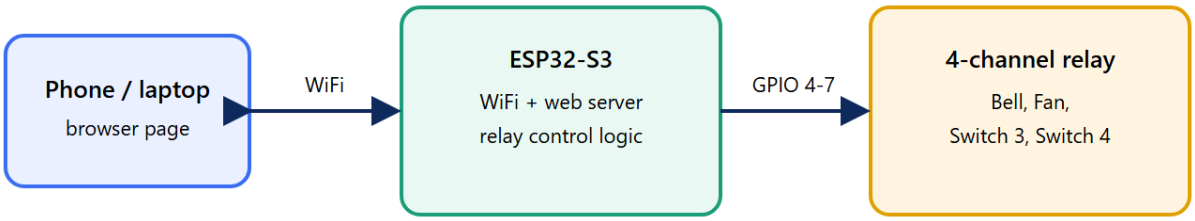


Figure 1. Block diagram of the system

The phone sends simple web requests to the ESP32-S3 over WiFi. The ESP32-S3 sets one of its output pins high or low. The pin drives a relay input, and the relay's contacts switch the load on or off. The load's power never passes through the ESP32.

Network modes

The board runs WiFi in two roles at once:

- **Own network (always on):** it creates a network named `HomeControl`. The phone joins it and opens `192.168.4.1`. No router is needed.
- **Joining a router:** the board tries the WiFi networks stored in its list, one after another, until one accepts it. It then gets an address from that router and prints the address on the serial monitor.

5. Circuit connections

Relay module pin	ESP32-S3 pin	Device
VCC	5V	Relay supply
GND	GND	Common ground
IN1	GPIO 4	Bell
IN2	GPIO 5	Fan
IN3	GPIO 6	Switch 3 (spare)
IN4	GPIO 7	Switch 4 (spare)

The strapping pins (GPIO 0, 3, 45, 46), the USB pins (19, 20) and the flash pins were avoided when choosing GPIO 4 to 7.

Connecting a load to a relay

Each relay has three screw terminals: **COM** (common), **NO** (normally open) and **NC** (normally closed). The loads are wired to COM and NO, so they stay off until the relay is activated.

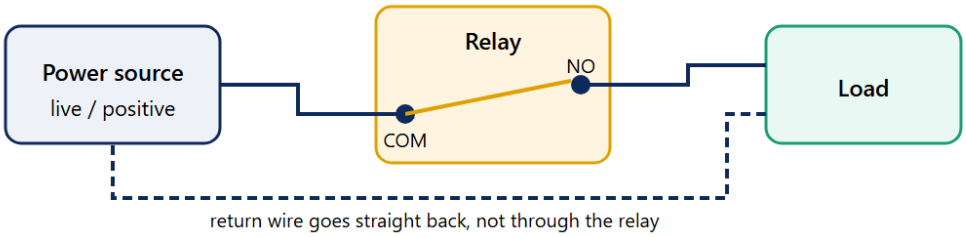


Figure 2. Wiring one load through a relay (COM and NO)

6. Software design

6.1 Firmware structure

The firmware is a single file, `src/main.cpp`. It contains:

- A **device list** that gives each switch a name, an icon, a GPIO pin and, for the bell, a ring time.
- **Relay handling:** pins are set as outputs and switched off at start-up, so no device turns on by itself after a power cut.
- **WiFi management:** starts the board's own network, then tries the stored networks one by one every 15 seconds until it is connected, and reconnects if the connection is lost.
- A small **web server** with the control page and four request handlers.
- A **timer** in the main loop that switches the bell off when its ring time has passed, without stopping the web server.

6.2 Web interface

The control page is stored inside the firmware and sent to the browser when it opens the board's address. It shows:

- One tile per device with a drawn icon (bell, fan, power). The fan icon spins while the fan is on, and the bell icon shakes while it rings.
- A toggle on each tile for the fan and the two spare switches, and a "1.5 SEC" tag on the bell tile.
- An "All off" button, and a text box for typed commands.
- A microphone button for voice commands.

The page asks the board for the current state every three seconds, so changes made from another phone also appear.

6.3 Request interface

Request	Action
`/`	Sends the control page.
`/api/state`	Returns the list of devices and their on/off state.
`/api/set?id=1&state=1`	Switches device id on (1) or off (0). The bell can be switched off but never held on.
`/api/ring?id=0`	Rings the bell for its ring time, then switches it off.
`/api/alloff`	Switches every device off.

6.4 Voice and typed commands

Speech is recognised by the phone's browser, which passes the recognised text to a small parser in the page. The parser looks for a device word (bell, ring, fan, or "switch" followed by a number) and an action word (on, off, stop). "Everything off" switches all devices off. The bell ignores "on" and always rings for its fixed time. The same parser handles typed text, which also lets a phone's keyboard dictation be used.

> **Voice control was tested on the phone and worked properly.** Browsers treat the microphone as a protected feature, so the first use may ask for microphone permission. The typed command box works in any browser and is available as a fallback.

7. Working of the system

1. The board is powered. It switches all relays off, starts its own network and begins searching for a stored WiFi network.
2. The phone joins either the board's own network or the same router as the board and opens the board's address.
3. The board sends the page. The page fetches the device state and draws the tiles.
4. The user taps a tile or gives a spoken or typed command. The page sends a request such as /api/set?id=1&state=1.
5. The board changes the GPIO pin level. Because the relay module is active-LOW, a LOW level switches the relay on.
6. The board replies with the new state and the page updates the tile.
7. For the bell, the board records the time to switch off. The main loop switches the relay off when that time arrives.

8. Testing and results

Test	Result
Firmware builds and uploads over USB	Passed. The memory used is about 30% of flash.
Board creates its own WiFi network and serves the page	Passed. The page loaded on the phone and laptop at 192.168.4.1.
Relays switch from the page	Passed. Relay LEDs and clicks followed the buttons.
Bell switches itself off after its ring time	Passed. Measured in a test: on at 4 s, off by 6 s for a 5 s setting; logs showed the switch-off exactly one ring time after each press.
Bell cannot be held on	Implemented in the firmware; the on request is ignored for the bell.
Voice commands on a phone	Passed. Voice control worked properly on the phone. Typed commands are available as an alternative.
Board joins a laptop hotspot and a college WiFi network	Passed when the board was within about a metre of the router or laptop. The page answered from the laptop on the same network.
Board joins a router several metres away	Did not work reliably. The board reported "no access point found".

9. Problems faced and solutions

Problem	Cause and solution
Board not detected by the computer	The cable carried power only. A data cable was needed.
UART port showed a driver error	The CP2102N USB-to-serial chip needs a driver. The board's other USB-C port connects directly to the ESP32-S3 and needs none, so that port was used.
Upload failed with "No serial data received"	The board was not in download mode. Holding BOOT while pressing RST fixed it; later uploads worked automatically.
All relays switched on at power-up	The start-up code set the pin level before making the pin an output, so it was ignored. The pins are now made outputs first and then set off.
Relays worked backwards	The relay module is active-LOW (LOW switches it on). The logic constants were set to match.
The phone hotspot would not connect	It broadcast only on 5 GHz, which the ESP32-S3 cannot use. The board's own network and a laptop hotspot set to 2.4 GHz were used.
Old bell symbol appeared as an empty box	The character was not supported on the phone. It was replaced with a drawn (vector) icon.
Board would not join distant routers	The board's WiFi reception is weak: it joined only at close range. Placing the board near the router solved it for the college network.

10. Limitations

- **No login:** the page has no password. Anyone on the same network who knows the board's address can use it. A PIN was built and tested, then removed at the user's request.
- **Local only:** it works only on the same network. Control from outside would need an internet service, which this project does not use.
- **WiFi range:** the board's reception is weak, so it must be placed close to the router.
- **Voice:** recognition is done by the phone's browser, so it needs a supported browser and microphone permission.
- **Address changes:** a router may give the board a different address after a restart.

11. Safety

Mains voltage is dangerous. If a load runs on mains power, wire it only through the relay's COM and NO terminals, cut or open only the live wire, switch the power off before wiring, and mount the relay module in an insulated enclosure. Check that the load's current is below the relay's rating (printed on the relay, commonly 10 A at 250 V AC). Test with a low-voltage load first.

12. Future scope

- Add a PIN or login to the page.
- Add timers and schedules (for example "fan off after 30 minutes") with a real-time clock.
- Connect to an internet service (such as MQTT) for control from anywhere, and for voice assistants.
- Add physical push buttons, and sensors such as temperature or motion.

- Allow wireless firmware updates, so the USB cable is not needed.
- Use a board with an external antenna for better range.

13. Conclusion

The project meets its main aim: a phone controls four relays through a web page served by an ESP32-S3, with a bell that rings for a fixed time, working voice and typed commands, and an own-network mode that works with no router. The control logic and web interface run entirely on the board. The main limits are the weak WiFi reception at a distance and the lack of a login, and both have clear solutions listed in the future scope.